THE COMPLETE CHEAT SHEET · 2025

Claude Code for Salesforce

Everything SF Admins, Developers & Architects need to transform from Trailhead assistant to autonomous DevOps engine.

From CLI Novice → Agentic SF Developer

9 DENSE SLIDES
50+ commands, real examples & SF-specific patterns

SFDX + MCP SETUP
Official DX MCP server, 60+ tools, community servers

AGENT WORKFLOWS
Multi-agent swarming, TDD, parallel instances, CI/CD

SKILLS & CLAUDE.MD
Project memory, subagents, hooks, custom commands

WHAT'S INSIDE

01 Why Claude Code
02 SF Capabilities
03 Dev Workflow
04 MCP Connections
05 Commands & Keys
06 Skills / CLAUDE.md
07 Prompting
08 Agent Loop
09 Getting Started

QUICK START

Install: `npm i -g @anthropic-ai/claude-code`

Run: `cd my-sfdx-project && claude`

Requires: Node.js 18+ · SFDX CLI · Auth'd Dev Hub · Claude Pro/Max/API

Platforms: macOS 13+ · Ubuntu 20+ · WSL · VS Code · Terminal · Neovim

01

Why Claude Code Changes Everything for SF Devs

	STANDARD ADMIN	CLAUDE CODE + SFDX
Metadata	✕ Manual UI edits	✓ Full XML read/write
UI setup	✕ Click-heavy config	✓ Script everything via CLI
Execution	✕ Isolated, manual runs	✓ Chained multi-step deploys
Build scope	✕ You build everything	✓ AI teammates build & test
Context	✕ Lost between tasks	✓ CLAUDE.md project memory
Tool chain	✕ Manual ALM switching	✓ MCP: GitHub, Jira, Slack
Testing	✕ Manual test writing	✓ Auto-gen tests + run
Code review	✕ Line-by-line manual	✓ AI reviews + PMD scan
Git ops	✕ Manual commits/PRs	✓ Auto branch + commit + PR
Onboarding	✕ Weeks to learn codebase	✓ Instant codebase mapping

WHAT IT ACTUALLY IS

An **agentic coding system** in your terminal. Reads codebase, runs SF CLI, modifies metadata, manages git, connects to 200+ tools via MCP, delegates to subagents — all via natural language.

NOT JUST A CHATBOT

It runs `sf` commands, edits Apex files, creates LWC components, writes test classes, manages scratch orgs, and commits to git — **all autonomously**.

PRICING: PRO
$20/mo
Standard usage

PRICING: MAX 5X
$100/mo
Heavy usage

PRICING: MAX 20X
$200/mo
Power user

API
Pay/token
Best for CI/CD

Models: Haiku · Sonnet · Opus

Requires: Node.js 18+ · SFDX CLI · Dev Hub · Git

IDEs: VS Code · IntelliJ · Terminal · Neovim · CI/CD

02

What Claude Code Can Do (SF Edition)

FULL METADATA ACCESS

Read/write force-app, analyze object XML, modify profiles, permission sets, page layouts. Map entire org shape.

Try: "Audit all custom objects and list fields with no description"

SFDX & TOOLING EXECUTION

Run `sf org create`, deploy, execute SOQL/SOSL, run Apex tests, push/pull source, open org.

Try: "Create scratch org, push source, run all tests"

MCP CONNECTIONS (200+ TOOLS)

GitHub, Jira, Slack, Postman, Sentry, Gmail, Notion — unified ALM chain interface.

Try: "Create a Jira ticket for this bug and link the PR"

MULTI-AGENT TEAMS

Spawn parallel agents: one writes LWC, another Apex, a third generates tests, fourth documents.

Try: "Create a team of 3 to build this feature"

ADMIN USE CASES

- Profile & perm set cleanup
- Field-level security audit
- Validation rule optimization
- Release notes summarization
- Duplicate rule analysis
- Layout comparison across profiles
- Sharing rule audit & cleanup

DEVELOPER USE CASES

- Apex trigger + handler gen
- LWC component scaffolding
- Flow-to-Apex migration
- Test class gen (90%+ cov)
- SOQL query optimization
- Aura → LWC conversion
- Batch / Queueable Apex

ARCHITECT USE CASES

- Data model review & ERD
- Integration pattern design
- Technical design docs
- Governor limit analysis
- Dependency mapping
- Security review (CRUD/FLS)
- API usage optimization

READS	WRITES	EXECUTES	MANAGES
Objects, Fields, Profiles, Flows, Apex, LWC, Perms	Apex, LWC, Flows, Tests, Triggers, Val Rules	SFDX CLI, SOQL, SOSL, Apex Anon, Test Runs	Git, Scratch Orgs, Deploys, PRs, CI/CD

03

The Salesforce Claude Code Workflow

ANALYZE

Audit org metadata
Review requirements
Summarize releases
Map dependencies
Check gov limits

›

PLAN

Draft tech design
Migration strategy
Object architecture
TDD test plan
Data model review

›

EXECUTE

Generate Apex/LWC
Build complex Flows
Write val rules
Create unit tests
Test data factory

›

SCALE

Set up CI/CD
Script data seeding
Create CLI plugins
Automate refreshes
Reusable skills

Jira Requirement → User Story → Apex + Test Generated → PR Created → Deployed to Scratch Org → Validated ✓

PRO WORKFLOW PATTERNS

TDD WITH CLAUDE CODE

1. Ask Claude to write a failing test first
2. Run it → see it fail
3. Claude writes code to make it pass
4. Refactor together
5. Repeat for next method

"Write failing test for AccountService.merge()"

PARALLEL INSTANCES

Run 3+ Claude Code sessions simultaneously:

- Terminal 1: Apex trigger + handler class
- Terminal 2: LWC UI component
- Terminal 3: Test classes & data factory
- Terminal 4: Documentation & PRs

Each session has its own context window

CI/CD AUTOMATION (HEADLESS)

```
claude -p "Review code" --output-format json --max-turns 3 >
```

PIPE CLI OUTPUT INTO CLAUDE

```
sf apex run test -r json | claude "Fix failures"
```

04

Connect Everything — MCP for Salesforce

Model Context Protocol — "USB-C for AI: one protocol, hundreds of connections."

DEVELOPER TOOLS

- GitHub
- Sentry
- GitLab
- Linear

PRODUCTIVITY

- Jira
- Asana
- Notion
- Slack

DATA & COMMS

- Gmail
- PostgreSQL
- Salesforce
- Perplexity
- Brave

OFFICIAL SF DX MCP SERVER

60+ tools: **data, metadata, lwc, apex, aura, mobile**. Reads real org schema. No hallucinated fields.

COMMUNITY MCP SERVERS

@tsmztech/mcp-server-salesforce — CRUD, SOSL, Apex

codefriar/sf-mcp — wraps SF CLI as MCP

Composio — 850+ apps single endpoint

DX TOOLSETS (--TOOLSETS FLAG)

data Query/DML	metadata Deploy	lwc Components
apex Classes	aura Migrate	all ⚠ 60+ tools

SETUP COMMANDS (COPY & PASTE)

```
# 1. Auth your org first
$ sf org login web -a myOrg

# 2. Official SF DX MCP
$ claude mcp add sf-dx -- npx -y \
  @salesforce/cli-mcp --orgs=default

# 3. Community SF MCP
$ claude mcp add salesforce -- npx \
  -y @tsmztech/mcp-server-salesforce

# 4. SF CLI wrapper
$ claude mcp add sf -- npx -y \
  codefriar/sf-mcp /path/to/project

# 5. GitHub integration
$ claude mcp add github -- npx -y \
  @modelcontextprotocol/server-github

# 6. Verify all connections
$ claude mcp list
```

--orgs: default | devhub | all | alias

MuleSoft: Anypoint Platform can convert any API to an MCP server

05

Essential Commands & Shortcuts

SLASH COMMANDS (TYPE / IN SESSION)

`/init`	Generate CLAUDE.md project memory
`/clear`	Wipe memory — hard reset context
`/compact`	Squeeze history — save tokens ★
`/model`	Switch: Haiku ↔ Sonnet ↔ Opus
`/cost`	Show burn rate — track $$ spent
`/review`	AI-powered code review
`/mcp`	Check MCP server connections
`/doctor`	Diagnose CLI issues — fix auth
`/config`	Settings & permissions config
`/permissions`	Manage tool approval rules
`/pr`	Create PR via GitHub CLI
`/bug`	Report issue to Anthropic
`/help`	Show all cmds + custom ones

CUSTOM SLASH COMMANDS

```
# .claude/commands/deploy-sit.md
---
description: Deploy to SIT
---
Deploy $ARGUMENTS to SIT. Run tests.
Report failures. If clean, create PR.
```

Share: `git add .claude/ && commit` — team-wide

FILE REFERENCES (@ TO REF)

`@filename`	@AccountTrigger.cls
`@folder/`	@force-app/main/lwc/
`@*.ext`	@*.trigger (all triggers)
`Tab`	Autocomplete metadata paths
`!`	Shell: `!sf org list`

KEYBOARD SHORTCUTS

Cancel / Stop	`Esc`
Rewind checkpoint	`Esc ×2`
Paste image	`Ctrl + V`
Auto-accept edits	`Shift + Tab`
Interrupt thinking	`Ctrl + C`
Prompt history	`↑`

CLI LAUNCH FLAGS

`-c`	Resume last session
`-p "prompt"`	One-shot headless mode
`--verbose`	Raw debug logs
`--model opus`	Use Opus model
`--output-format json`	Machine-readable output
`--max-turns 5`	Limit agent loop iterations
`--allowedTools`	Restrict available tools
`--dangerously-skip-permissions`	GOD MODE ⚠

06

Skills & CLAUDE.md — Project Memory

CLAUDE SKILLS

Task-specific instructions auto-loaded per project. SKILL.md with templates, scripts. Create once, reuse everywhere.

CLAUDE.MD

Markdown at project root — always loaded. Tech stack, conventions, key commands. **#1 most important file** for Claude Code.

SKILLS FOLDER

```
~/.claude/skills/
├── apex-refactorer/ SKILL.md
├── flow-documenter/ SKILL.md
├── profile-cleaner/ SKILL.md
├── test-data-factory/ SKILL.md
└── lwc-generator/ SKILL.md
```

SETTINGS PRIORITY

1. Enterprise managed-settings.json
2. Project .claude/settings.json
3. Local .claude/settings.local.json
4. User ~/.claude/settings.json

SUBAGENTS (SPECIALISTS)

```
# .claude/agents/sf-reviewer.md
---
name: sf-reviewer
model: sonnet
tools: Read, Grep, Glob, Bash
---
Expert SF code reviewer. Focus on
security, bulkification, CRUD/FLS,
and PMD compliance.
```

HOOKS (AUTO-RUN)

```
# Auto-PMD on Apex save
"hooks": {"PostToolUse": [{
  "matcher":"Write(*.cls)",
  "hooks":[{"type":"command",
   "command":"pmd check $file"}]
}]}
```

EXAMPLE CLAUDE.MD FOR SF

```
# Project: Service Cloud Implementation

## Tech Stack
LWC, Apex, Flow, OmniStudio, FSL
Certinia PSA for project management

## Key Commands
sf project deploy start -o sit
sf apex run test -n MyTest -r human
sf org open -o dev
sf data query -q "SELECT Id FROM Account"

## Coding Standards
- Follow PMD rules. No hardcoded IDs
- Trigger handler pattern (1 trigger/obj)
- Bulkify all — no SOQL in loops
- 90%+ test coverage required
- LWC: base components only, no 3rd party
- Always check CRUD/FLS before DML
- Custom Metadata (not Custom Settings)

## Naming Conventions
- Triggers: [Object]Trigger.trigger
- Handlers: [Object]TriggerHandler.cls
- Services: [Domain]Service.cls
- Tests: [Class]Test.cls
- LWC: camelCase folder, PascalCase class

## Architecture Decisions
- Service layer for complex logic
- Platform events for async processing
- Custom metadata for all config
- Named credentials for callouts
```

07

Prompting Techniques for Salesforce

Be Specific	"Create 'After Update' Trigger on Contact to update parent Account. Handle recursion."
Examples	"Write test class following 'testSetup' pattern in @AccountTriggerTest.cls"
Chain Steps	"First analyze Flow, then summarize, then optimize."
Constraints	"LWC base only. Pass PMD. No SOQL in loops. Check CRUD/FLS."
Role Play	"Act as SF Technical Architect reviewing data model."
Plan First	"Plan WFR → Flow migration. Review. Then generate."

POWER PATTERNS

CHECKPOINT + ITERATE

1. Ask Claude to plan first
2. Review (don't code yet)
3. Checkpoints auto-created
4. Rewind (`Esc` ×2) if wrong
5. Iterate with specific feedback

CONTEXT LOADING

1. Files: `@AccountTrigger.cls`
2. Folders: `@force-app/`
3. Pipe: `sf data query | claude`
4. Screenshots: `Ctrl+V`
5. ERD images, UI mocks, error pages

READY-TO-USE SF PROMPT TEMPLATES

🔍 CODE REVIEW

"Review @force-app/main/ for:
1. Bulkification (SOQL in loops)
2. Missing null checks
3. Hardcoded IDs / record types
4. Missing CRUD/FLS checks
5. Test coverage gaps
6. Governor limit risks"

📦 MIGRATION

"Analyze all WFRs on Case. For each:
1) Document logic 2) Design Flow
3) Generate metadata 4) Test plan"

🏗 FEATURE BUILD

"Build case auto-assignment:
1. Trigger on Case (after insert)
2. Handler with routing logic
3. Custom metadata for rules
4. Test class 95%+ coverage
5. Deploy + run all tests"

🏥 ORG HEALTH CHECK

"Audit: unused fields, profiles with Modify All, WFRs to migrate, API usage"

📊 DATA MODEL

"ERD for Account-related objects. Show lookups, MD, junctions. Flag deps."

08 + 09

Agentic Loop, Swarming & Getting Started

THE 4-PHASE AGENT LOOP

1. CONTEXT INJECT
Load: `@src/auth` · Pipe: `cat log | claude "Fix"`
Paste screenshots, ERDs, UI mocks via `Ctrl+V`

2. PLAN MODE
"Create migration plan. Don't code yet."
Forces structured thinking. Review before executing.

3. EXECUTION
"Implement step 1. Run tests." Writes code, runs `sf` commands, auto-iterates on failure.

4. VERIFY & COMMIT
"If green, commit 'feat: auth'." Self-corrects. Runs PMD. Creates branch + PR automatically.

AGENT SWARMING (MULTI-AGENT)

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Leader orchestrates → Worker 1: LWC/Aura
→ Worker 2: Apex/Flow → Worker 3: Tests/Docs
→ Delegate: Ephemeral one-off tasks

Spawn: "team of 3 to build this"

View: `/agents` · tmux splits

EXTENDED THINKING

Enabled by default. Claude reasons before coding. Controlled via settings. Charged per thinking token.

PRO CONFIG

`BASH_MAX_TIMEOUT_MS` — long deploys
`MCP_TOOL_TIMEOUT` — large SOQL
`preferredNotifChannel` — alerts

GETTING STARTED (5 STEPS)

1. Install VS Code + SF Extensions
2. `npm i -g @anthropic-ai/claude-code`
3. Open your SFDX project folder
4. Terminal → `claude` → authenticate
5. Run `/init` to gen CLAUDE.md

Node.js 18+ · **SFDX CLI** · **Dev Hub** · **Claude Pro/Max/API** · **macOS/Ubuntu/WSL** · **Git**

FIRST 5 THINGS TO TRY

```
"Explain the architecture of this project"
"List all custom objects and field counts"
"Create a test for @AccountTrigger.cls"
"Review @force-app/ for PMD violations"
"Push to scratch org and run all tests"
```

RESOURCES

`docs.claude.com` — Official docs
`github.com/salesforcecli/mcp` — SF DX MCP
`awesome-claude-code` — Community tools
Skills, hooks, agents, plugins directory

🚀 **Save & Share:** Upload this PDF as a LinkedIn document post for a swipeable carousel!